PRG 107 – Python Programming Kristiania University of Applied Sciences Lec7

By Huamin Ren

Huamin.ren@kristiania.no

Outline - Advanced topics in Python II

1. Introduction to Numpy
2. The Numpy ndarray
3. Fast element-wise array functions

Outline - Advanced topics in Python I

1. *Introduction to Numpy*
2. The Numpy ndarray
3. Fast element-wise array functions

1. Introduction to Numpy

Numerical Python

- ***Ndarray***: an efficient multidimensional array providing fast array-oriented arithmetic operations and flexible broadcasting capabilities
- ***Mathematical functions*** for fast operations on entire arrays of data without having to write loops
- ***Tools for reading/writing array data to disk*** and working with memory-mapped files.
- ***Linear algebra***, ***random number generation***, and ***Fourier transform capabilities***

Outline - Advanced topics in Python I

1. Introduction to Numpy
2. *The Numpy ndarray*
3. Fast element-wise array functions

2. Ndarray

- N-dimensional array object, or ndarray
- Able to perform mathematical operations on whole blocks of data using similar syntax to the equivalent operations between scalar elements

2. Ndarray

- All of the elements must be the same type

`shape`, a tuple indicating the size of each dimension

```
#%%
print(type(data))  data: {ndarray: (2, 3, 2)}
print(data.ndim)  data: {ndarray: (2, 3, 2)}
print(data.shape)  data: {ndarray: (2, 3, 2)}
print(type(data[0,0,0])==type(data[0,0,1]))
```

`dtype`, an object describing the data type of the array

2.1 Creating ndarrays

- Use the array function
- Use other functions to create ndarrays

```
#%%
print(np.zeros(10))
print(np.zeros((3, 6)))
print(np.empty((2, 3, 2)))
```

```
data2 = [[1, 2, 3, 4], [5, 6, 7, 8]]
arr2 = np.array(data2)  data2: [[1, 2, 3, 4],
print(type(data2))  data2: [[1, 2, 3, 4], [5,
print(type(arr2))  arr2: {ndarray: (2, 4)}
```

Table 4-1. Array creation functions

Function	Description
`array`	Convert input data (list, tuple, array, or other sequence type) to an ndarray either by inferring a dtype or explicitly specifying a dtype; copies the input data by default
`asarray`	Convert input to ndarray, but do not copy if the input is already an ndarray
`arange`	Like the built-in `range` but returns an ndarray instead of a list
`ones`, `ones_like`	Produce an array of all 1s with the given shape and dtype; `ones_like` takes another array and produces a ones array of the same shape and dtype
`zeros`, `zeros_like`	Like `ones` and `ones_like` but producing arrays of 0s instead
`empty`, `empty_like`	Create new arrays by allocating new memory, but do not populate with any values like `ones` and `zeros`
`full`, `full_like`	Produce an array of the given shape and dtype with all values set to the indicated "fill value" `full_like` takes another array and produces a filled array of the same shape and dtype
`eye, identity`	Create a square N × N identity matrix (1s on the diagonal and 0s elsewhere)

Question: select several functions from above table, and test them through the implementation.

2.2 Data Types for ndarrays

- Data type or dtype

A special object containing the information (or metadata, data about data) the ndarray needs to interpret a chunk of memory as a particular type of data

dtypes are a source of NumPy's flexibility for interacting with data coming from other systems. In most cases they provide a mapping directly onto an underlying disk or memory representation, which makes it easy to read and write binary streams of data to disk and also to connect to code written in a low-level language like C or Fortran.

2.2 Data Types for ndarrays

```
# 2.3.2 convertions between dtypes
arr = np.array([1, 2, 3, 4, 5])
print(arr.dtype)  arr: {ndarray: (5,)}
float_arr = arr.astype(np.float64)  arr: {ndar
float_arr.dtype  float_arr: {ndarray: (5,)}
```

```
arr1 = np.array([1, 2, 3], dtype=np.float64)
arr2 = np.array([1, 2, 3], dtype=np.int32)
arr1.dtype  arr1: {ndarray: (3,)}
arr2.dtype  arr2: {ndarray: (3,)}
```

2.2 Data Types for ndarrays

Table 4-2. NumPy data types

Type	Type code	Description
`int8, uint8`	`i1, u1`	Signed and unsigned 8-bit (1 byte) integer types
`int16, uint16`	`i2, u2`	Signed and unsigned 16-bit integer types
`int32, uint32`	`i4, u4`	Signed and unsigned 32-bit integer types
`int64, uint64`	`i8, u8`	Signed and unsigned 64-bit integer types
`float16`	`f2`	Half-precision floating point
`float32`	`f4 or f`	Standard single-precision floating point; compatible with C float
`float64`	`f8 or d`	Standard double-precision floating point; compatible with C double and Python `float` object
`float128`	`f16 or g`	Extended-precision floating point
`complex64, complex128, complex256`	`c8, c16, c32`	Complex numbers represented by two 32, 64, or 128 floats, respectively
`bool`	?	Boolean type storing `True` and `False` values
`object`	O	Python object type; a value can be any Python object
`string_`	S	Fixed-length ASCII string type (1 byte per character); for example, to create a string dtype with length 10, use `'S10'`
`unicode_`	U	Fixed-length Unicode type (number of bytes platform specific); same specification semantics as `string_` (e.g., `'U10'`)

2.2 Data Types for ndarrays

- ***astype***: convert or cast an array from one dtype to another

```
int_array = np.arange(10)
print(int_array)  int_array: {ndarray: (10,)}
calibers = np.array([.22, .270, .357, .380, .44, .50], dtype=np.float)
print(calibers)  calibers: {ndarray: (6,)}
# comment on the new variable
int_array_new = int_array.astype(calibers.dtype)  int_array: {ndarray:
print(int_array_new)  int_array_new: {ndarray: (10,)}
print(type(int_array_new[0]))  int_array_new: {ndarray: (10,)}
```

2.3 Arithmetic with NumPy Arrays

- *Vectorization*: able to perform batch operations on data without writing any for loops
- Any arithmetic operations between *equal-size arrays* applies the operation *element-wise*

2.4 Basic Indexing and Slicing

- *Many ways* to select a subset of your data
- Array slices are *views* on the original array

NumPy has been designed to be able to work with very large arrays, you could imagine performance and memory problems if NumPy insisted on always copying data.

Particularly understand 2-d data

- Arr[1][0]
- Arr[1,0]

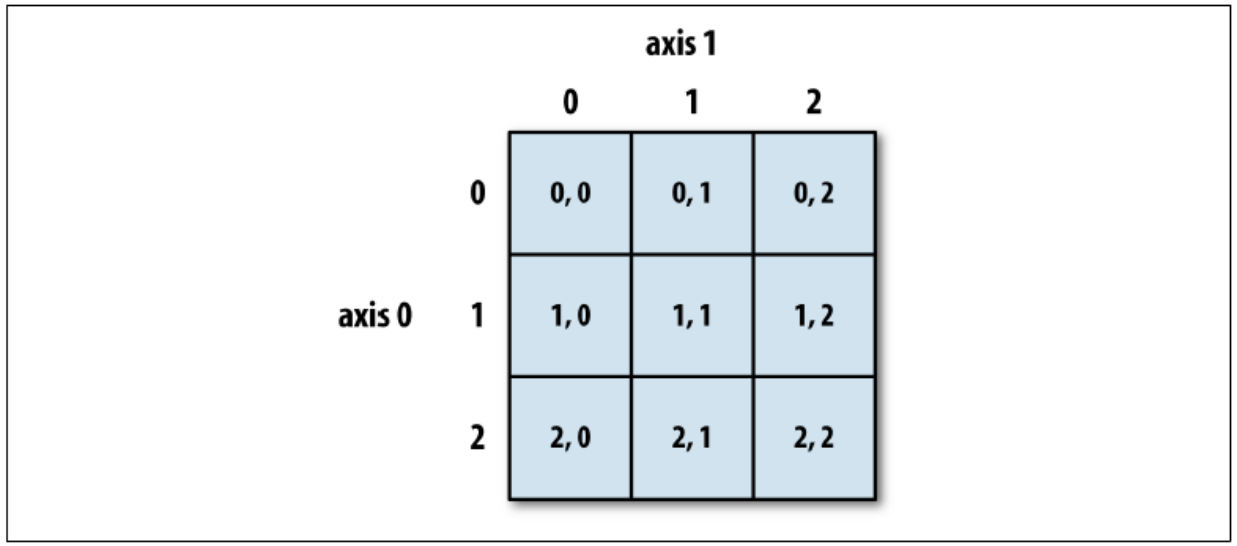


Figure 4-1. Indexing elements in a NumPy array

```
In [76]: arr3d = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

In [77]: arr3d
Out[77]:
array([[[ 1,  2,  3],
        [ 4,  5,  6]],
       [[ 7,  8,  9],
        [10, 11, 12]]])
```

`arr3d[0]` is a 2×3 array:

```
In [78]: arr3d[0]
Out[78]:
array([[1, 2, 3],
       [4, 5, 6]])
```

?array3d
?array3d[0]
?array3d[1]
?array3d[0,0]
?array3d[0,0,0]

Indexing with slices

- 1-d slicing
- 2-d slicing

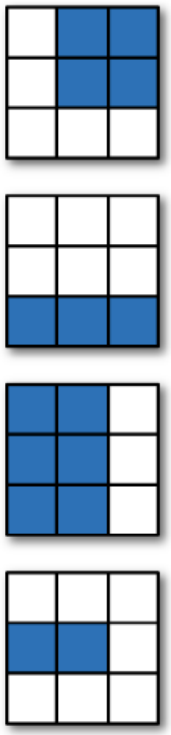
```
In [90]: arr2d
Out[90]:
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

```
In [91]: arr2d[:2]
Out[91]:
array([[1, 2, 3],
       [4, 5, 6]])
```

```
In [92]: arr2d[:2, 1:]
Out[92]:
array([[2, 3],
       [5, 6]])
```

```
In [93]: arr2d[1, :2]
Out[93]: array([4, 5])
```

```
In [95]: arr2d[:, :1]
Out[95]:
array([[1],
       [4],
       [7]])
```



Expression	Shape
`arr[:2, 1:]`	`(2, 2)`
`arr[2]`	`(3,)`
`arr[2, :]`	`(3,)`
`arr[2:, :]`	`(1, 3)`
`arr[:, :2]`	`(3, 2)`
`arr[1, :2]`	`(2,)`
`arr[1:2, :2]`	`(1, 2)`

2.5 Boolean Indexing

```
array([[-1.34483761,  0.21312328, -0.43670183,  0.61926796],
       [ 0.77134154, -0.6689614 , -0.7928281 , -0.70045122],
       [ 0.62966362, -0.02701028,  0.10702016, -0.28455877],
       [ 1.1737119 , -0.8918302 ,  0.45100106,  1.27928438],
       [-0.76866134,  0.39503586,  1.09501753,  0.13217849],
       [ 1.47289043,  0.77806889, -0.30034938,  0.15502661],
       [-0.77865714,  0.81174698, -0.10668945, -0.75979303]])
```

```
array([ True, False, False,  True, False, False, False])
```

```
array([[-1.34483761,  0.21312328, -0.43670183,  0.61926796],
       [ 1.1737119 , -0.8918302 ,  0.45100106,  1.27928438]])
```

2.6 Fancy Indexing

```
arr = np.empty((8, 4))
for i in range(8):
    arr[i] = i
arr
```

```
array([[0., 0., 0., 0.],
       [1., 1., 1., 1.],
       [2., 2., 2., 2.],
       [3., 3., 3., 3.],
       [4., 4., 4., 4.],
       [5., 5., 5., 5.],
       [6., 6., 6., 6.],
       [7., 7., 7., 7.]])
```

2.6 Fancy Indexing

```
arr[[-3, -5, -7]]
```

```
array([[5., 5., 5., 5.],
       [3., 3., 3., 3.],
       [1., 1., 1., 1.]])
```

```
arr[[4, 3, 0, 6]]
```

```
array([[4., 4., 4., 4.],
       [3., 3., 3., 3.],
       [0., 0., 0., 0.],
       [6., 6., 6., 6.]])
```

2.6 Fancy Indexing

```
arr = np.arange(32).reshape((8, 4))
arr
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23],
       [24, 25, 26, 27],
       [28, 29, 30, 31]])
```